

Homework 3

Please complete the following problems. You must submit a document containing the following for each problem:
a) the answer and b) Python code that can be run to verify your work.

1. Rabbits and Recurrence Relations

In 1202, Leonardo of Pisa (commonly known as Fibonacci) considered a mathematical exercise regarding the reproduction of a population of rabbits. He made the following simplifying assumptions about the population:

1. The population begins in the first month with a pair of newborn rabbits.
2. Rabbits reach reproductive age after one month.
3. In any given month, every rabbit of reproductive age mates with another rabbit of reproductive age.
4. Exactly one month after two rabbits mate they produce one male and one female rabbit.
5. Rabbits never die or stop reproducing.

Fibonacci's exercise was to calculate how many pairs of rabbits would remain in one year. We can see that in the second month, the first pair of rabbits reach reproductive age and mate. In the third month, another pair of rabbits is born, and we have two rabbit pairs; our first pair of rabbits mates again. In the fourth month, another pair of rabbits is born to the original pair, while the second pair reach maturity and mate (with three total pairs). The dynamics of the rabbit population are illustrated in Figure 1. After a year, the rabbit population boasts 144 pairs.

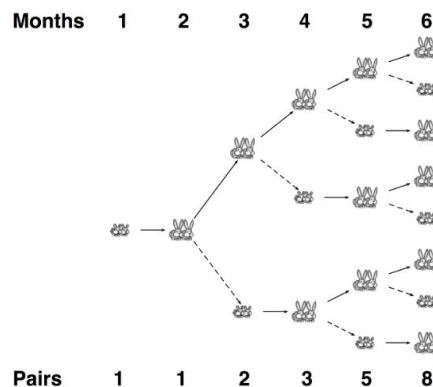


Figure 1: The growth of Fibonacci's rabbit population for the first six months.

Although Fibonacci's assumption of the rabbits' immortality may seem a bit farfetched, his model was not unrealistic for reproduction in a predator-free environment: European rabbits were introduced to Australia in the mid-19th Century, a place with no real indigenous predators for them. Within 50 years, the rabbits had already eradicated many plant species across the continent, leading to irreversible changes in the Australian ecosystem and turning much of its grasslands into eroded, practically uninhabitable parts of the modern Outback (see Figure 2).



Figure 2: Erosion at Lake Mungo in New South Wales, which was initiated by European rabbits in the 19th Century. Courtesy Pierre Pouliquin.

In this problem, we will use the simple idea of counting rabbits to introduce the computational topic of building up large solutions from smaller ones.

PROBLEM

A sequence is an ordered collection of objects (usually numbers), which are allowed to repeat. Sequences can be finite or infinite. Two examples are the finite sequence $(\pi, -2 - \sqrt{0}, \pi)$ and the infinite sequence of odd numbers $(1, 3, 5, 7, 9, \dots)$. We use the notation a_n to represent the n -th term of a sequence.

A recurrence relation is a way of defining the terms of a sequence with respect to the values of previous terms. In the case of Fibonacci's rabbits from the introduction, any given month will contain the rabbits that were alive the previous month, plus any new offspring. A key observation is that the number of offspring in any month is equal to the number of rabbits that were alive two months prior. As a result, if F_n represents the number of rabbit pairs alive after the n -th month, then we obtain the Fibonacci sequence having terms F_n that are defined by the recurrence relation $F_n = F_{n-1} + F_{n-2}$ (with $F_1 = F_2 = 1$ to initiate the sequence). Although the sequence bears Fibonacci's name, it was known to Indian mathematicians over two millennia ago.

When finding the n -th term of a sequence defined by a recurrence relation, we can simply use the recurrence relation to generate terms for progressively larger values of n . This problem can be solved by the computational technique of dynamic programming, which successively builds up solutions by using the answers to smaller cases.

Given: A file containing positive integers $n \leq 40$ and $k \leq 5$.

Return: The total number of rabbit pairs that will be present after n months, if we begin with 1 pair and in each generation, every pair of reproduction-age rabbits produces a litter of k rabbit pairs (instead of only 1 pair).

Sample Dataset

5 3

Sample Output

19

2. RNA Splicing

In the “Transcribing DNA into RNA” problem, it was mentioned that a strand of DNA is copied into a strand of RNA during transcription, but there was no mention of how transcription is achieved.

In the nucleus, an enzyme (i.e., a molecule that accelerates a chemical reaction) called RNA polymerase (RNAP) initiates transcription by breaking the bonds joining complementary bases of DNA. It then creates a molecule called precursor mRNA, or pre-mRNA, by using one of the two strands of DNA as a template strand: moving down the template strand, when RNAP encounters the next nucleotide, it adds the complementary base to the growing RNA strand, with the provision that uracil must be used in place of thymine; see Figure 1.

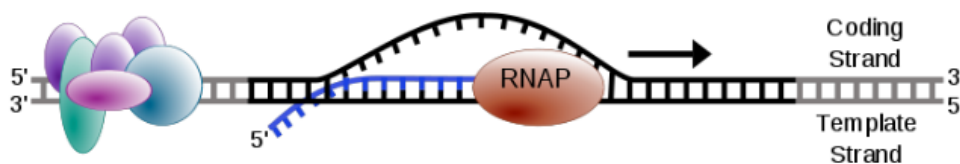


Figure 1: The elongation of a pre-mRNA by RNAP as it moves down the template strand of DNA.

Because RNA is constructed based on complementarity, the second strand of DNA, called the coding strand, is identical to the new strand of RNA except for the replacement of thymine with uracil. See Figure 2.

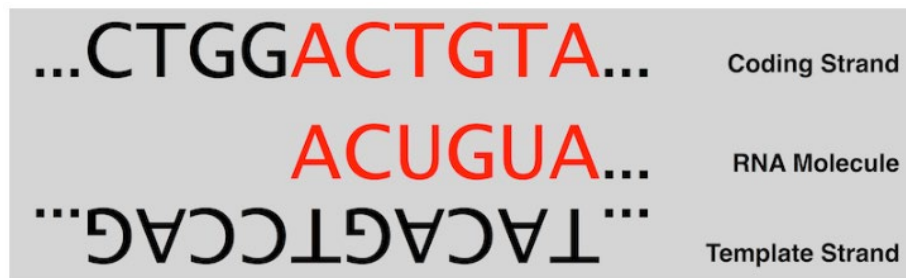


Figure 2: RNA is identical to the coding strand except for the replacement of thymine with uracil.

After RNAP has created several nucleotides of RNA, the first separated complementary DNA bases then bond back together. The overall effect is very similar to a pair of zippers traversing the DNA double helix, unzipping the two strands, and then quickly zipping them back together while the strand of pre-mRNA is produced.

For that matter, it is not the case that an entire substring of DNA is transcribed into RNA and then translated into a peptide one codon at a time. In reality, a pre-mRNA is first chopped into smaller segments called introns and exons; for the purposes of protein translation, the introns are thrown out, and the exons are glued together sequentially to produce a final strand of mRNA. This cutting and pasting process is called splicing, and it is facilitated by a collection of RNA and proteins called a spliceosome. The fact that the spliceosome is made of RNA and proteins despite regulating the splicing of RNA to create proteins is just one manifestation of a molecular chicken-and-egg scenario that has yet to be fully resolved.

In terms of DNA, the exons deriving from a gene are collectively known as the gene's coding region.



PROBLEM

After identifying the exons and introns of an RNA string, we only need to delete the introns and concatenate the exons to form a new string ready for translation.

Given: A file containing DNA string s (of length at most 1 kb) and a collection of substrings of s acting as introns. All strings are given in FASTA format.

Return: A protein string resulting from transcribing and translating the exons of s . (Note: Only one solution will exist for the dataset provided.)

Sample Dataset

```
>Rosalind_10
ATGGTCTACATAGCTGACAAACAGCACGTAGCAATCGGTCTCGAGAGGCATATGGTCACATGATCGGTGAGCGTGTTTCAAAGTTTGC
CGCCTAG
>Rosalind_12
ATCGGTCGAA
>Rosalind_15
ATCGGTCGAGCGTGT
```

Sample Output

```
MVYIADKQHVASREAYGHMFKVCA
```



3. Finding a Spliced Motif

In the “Finding a Motif in DNA” problem, you searched for occurrences of a motif as a substring of a larger database genetic string. However, because a DNA strand coding for a protein is often interspersed with introns, you need a way to recognize a motif that has been chopped up into pieces along a chromosome.

PROBLEM

A subsequence of a string is a collection of symbols contained in order (though not necessarily contiguously) in the string (e.g., ACG is a subsequence of TATGCTAAGATC). The indices of a subsequence are the positions in the string at which the symbols of the subsequence appear; thus, the indices of ACG in TATGCTAAGATC can be represented by (2, 5, 9).

As a substring can have multiple locations, a subsequence can have multiple collections of indices, and the same index can be reused in more than one appearance of the subsequence; for example, ACG is a subsequence of AACCGGT in 8 different ways.

Given: A file containing two DNA strings s and t (each of length at most 1 kb). Both strings are in FASTA format.

Return: One collection of indices of s in which the symbols of t appear as a subsequence of s . If multiple solutions exist, then you may return any solution.

Sample Dataset

```
>Rosalind_14
ACGTACGTGACG
>Rosalind_18
GTA
```

Sample Output

```
3 8 10
```

Hint: For the mathematically inclined, we may equivalently say that $t = t_1 t_2 \cdots t_m$ is a subsequence of $s = s_1 s_2 \cdots s_n$ if the characters of t appear in the same order within s . Even more formally, a subsequence of s is a string $s_{i_1} s_{i_2} \cdots s_{i_k}$, where $1 \leq i_1 < i_2 < \cdots < i_k \leq n$.



4. Finding a Shared Motif

In the “Finding a Motif in DNA” problem, you searched a given genetic string for a motif; however, this problem assumed that you knew the motif in advance. In practice, biologists often do not know exactly what they are looking for. Rather, they must hunt through several different genomes at the same time to identify regions of similarity that may indicate genes shared by different organisms or species.

The simplest region of similarity is a motif occurring without mutation in every one of a collection of genetic strings taken from a database; such a motif corresponds to a substring shared by all the strings. We want to search for long shared substrings, as a longer motif will likely indicate a greater shared function.

PROBLEM

A common substring of a collection of strings is a substring of every member of the collection. We say that a common substring is a longest common substring if there does not exist a longer common substring. For example, "CG" is a common substring of "ACGTACGT" and "AACCCGTATA", but it is not as long as possible; in this case, "CGTA" is a longest common substring of "ACGTACGT" and "AACCCGTA".

Note that the longest common substring is not necessarily unique; for a simple example, "AA" and "CC" are both longest common substrings of "AACC" and "CCAA".

Given: A file containing a collection of k ($k \leq 100$) DNA strings of length at most 1 kb each in FASTA format.

Return: A longest common subsequence of the collection. If multiple solutions exist, then you may return any one of the solutions.

Hint: The simplest approach starts with the realization that the longest common substring cannot be longer than the shortest string in the collection. You may begin by sorting the DNA strings by length and removing the shortest string, with length L , to a privileged position. This can be done by moving the string outside the list or moving it to position zero on the list. Next, you can test the substrings of the shortest string, beginning with the single substring of length L , then the two substrings of length $L-1$, and so forth. If you start with the longest possible substring candidates and only examine shorter substrings after eliminating the longer substrings, then you could stop the search as soon as you find a common substring to speed up the process.